Bandit - Level 09

Goal

The password for the next level is stored in `data.txt`, in **one of the few human-readable strings**, preceded by several `=` characters.

Connection

```
ssh -p 2220 bandit9@bandit.labs.overthewire.org
```

Use the password retrieved at the end of Bandit - Level 08.

Concepts

`data.txt` is **not a text file**. It's binary data with a handful of readable fragments embedded in it — which changes what the standard text tools can do with it.

`grep` refuses to print matches from binary input by default, reporting `binary file matches` instead. This is a **safety feature, not an error**: dumping raw binary to a terminal can emit control bytes that corrupt the session (change the charset, garble the display, leave the prompt unusable). `grep -a` forces it to treat the input as text and print anyway.

But forcing it isn't enough. `grep` operates on **lines**, and a line ends at a `\n` byte. In binary data those bytes occur at random, so the "line" holding the password drags hundreds of unreadable bytes along with it. The match is technically there — just not usable.

`strings` is the right tool: it scans a file byte by byte for runs of **printable characters at least 4 long** (default), prints each run as its own line, and discards everything else. It converts a binary problem into a text problem, at which point `grep` works normally again.

On inspecting before filtering. `strings` on this file emits hundreds of lines — far more than a terminal shows at once. Piping it through `head -30` caps the output at the first 30 lines, which is what makes it readable. This step solves nothing on its own; it exists to **verify the new tool behaves as expected before trusting it**. See Linux - Sampling and Inspecting Output.

Attempts

```
greep "=====" data.txt
# → Command 'greep' not found, did you mean: command 'grep' from deb grep (3.12-1)
# Typo. Ubuntu's command-not-found handler catches it — read the suggestion, don't
retype blind.

grep "=====" data.txt
# → grep: data.txt: binary file matches
# Not an error. grep confirms a match exists but withholds the output because the
input is binary.

base64 -d data.txt > data1.txt | grep "===="
```

```
# → -bash: data1.txt: Permission denied
# Two separate problems in one line:
#   1. The Bandit home directory is read-only — no writing files there.
#   2. base64 was the wrong tool anyway; this file is not base64-encoded.

chmod 777 data1.txt      # → No such file or directory (the redirect never created
it)
find data1.txt           # → No such file or directory (same reason)
ls                       # → data.txt   — only the original file exists

base64 -d data.txt
# → base64: error: invalid input
# Confirms it: the content is not valid base64. Wrong tool.

base64 -d data.txt | sort data.txt | grep "="
# → base64: error: invalid input
# → grep: (standard input): binary file matches
# `sort data.txt` takes a FILE ARGUMENT, so it ignores whatever arrives on stdin.
# The pipe from base64 was silently discarded.

grep -a "=====" data.txt
# → Works — the password IS visible at the end of the output, but buried in
#   hundreds of bytes of binary noise because grep returns whole LINES.
```

Solution

```
strings data.txt | head -30
# → inspect first: readable fragments, one per line, noise gone
```

`head -30` is **not part of the answer** — remove it and the level still solves. It's a reconnaissance step: `strings` was a tool used here for the first time, and this confirmed three things at a glance before committing to a filter. The noise was gone, the output was one string per line, and '========== the' was already visible — proving the target pattern existed and had the expected shape.

Without it, the next command would have been written blind. Had it returned nothing, there'd be no way to tell whether the fault was in `strings`, in the pattern, or in the assumption about the file.

```
strings data.txt | grep '==='
# → ========== the
#    ========== password
#    Y========== is
#    ========== B0s2khmbT9u0geKuOoVGW3JZKhndE3BG
```

The message is split across four separate printable runs. Reassembled: *the password is* `B0s2khmbT9u0geKuOoVGW3JZKhndE3BG`.

Three '=' in the pattern rather than five is deliberate — the number of leading '=' varies between fragments (note the `Y==========` line), so a looser pattern catches all of them.

Points of Friction

1. Read `binary file matches` as a failure. It was a report of success with the output withheld. The distinction matters: `grep` had already found the match on the very first correct command — the missing piece was how to *render* it, not how to *find* it.

2. Reached for `base64` before identifying the file. The tool list on the level page (`grep, sort, uniq, strings, base64, tr, tar, gzip, bzip2, xxd`) is a shared inventory for the whole 9→13 stretch, not a recipe for this level. `base64` belongs to Level 10, `tr` to Level 11, `xxd` / `gzip` / `bzip2` / `tar` to Level 12. One `file data.txt` up front would have settled it — same lesson already recorded in Linux - File Type Detection.

3. `base64 -d data.txt | sort data.txt | grep "="` — the pipe was dead. When a command receives a filename as an argument, it reads that file and **ignores stdin entirely**. The data coming down the pipe was thrown away silently, with no error. In a pipeline, every command after the first goes **without** a file argument. Third variant of this same mistake, after `find | grep` in Bandit - Level 07: see Linux - Piping and Redirection.

4. The Bandit home directory is read-only. `> data1.txt` failed with *Permission denied*, and because the redirect never created the file, the follow-up `chmod` and `find` failed too — cascading errors from one root cause. Scratch files go in `/tmp`, in a private subdirectory:

```
mkdir -p /tmp/b9david && cd /tmp/b9david
```

Key Takeaway

`strings` extracts the printable runs from a binary and emits them as normal lines, which is what makes every downstream text tool usable again. `grep -a` also technically works here, but returns whole binary "lines" — the answer is present yet unreadable. **Being able to find something and being able to read it are two different problems**, and `strings` solves the second.

The instinct worth keeping: when a text tool complains about binary input, the answer is almost never to force it (`-a`) — it's to convert the input to text first. Full treatment, including the flags that matter in malware analysis and forensics: Linux - Extracting Strings from Binaries.

Second habit, carried over from Bandit - Level 08 where it was applied unprompted: **look at the data before filtering it.** In Level 8 that was `wc -l` and `uniq -c` to map the file's structure; here it was `head -30` to confirm what an unfamiliar tool produces. Neither step contributes to the answer — both make the answer trustworthy.

Next

```
ssh -p 2220 bandit10@bandit.labs.overthewire.org
```

◀ Previous: Bandit - Level 08 · Next ▶ Bandit - Level 10